

A Judge That Cannot Certify Itself: A Code-Enforced Trust Protocol for Adversarial LLM Auditing

Anonymous Author(s)
Affiliation withheld for double-blind review
email withheld

Abstract

LLM-based evaluators fail most dangerously when they are *reliable without being valid*: internally consistent, confident, and biased. The largest systematic study of LLM-as-a-judge to date reports exactly this dissociation—test–retest consistency above 0.95 coexisting with severe position bias, and exact-match agreement overstating discrimination by 33–41 points of κ once chance-corrected [1]. Such a judge is at its most harmful the moment it is allowed to *certify itself*—to declare its own output validated, or to count its own internal agreement as independence. We present a *trust protocol*: a small set of invariants an adversarial LLM-audit pipeline is made to obey *in ordinary code*, not in prompts, and each pinned by a test. (1) The pipeline can never return `VALIDATED` on internal grounds; human closure is a *cryptographic* HMAC of the audit-ledger digest under a key the model cannot reach, so a model that authors the payload cannot sign its own validation. (2) Cross-model independence is *attested by the calling adapter*, not read from a self-report. (3) A run is admitted only under an *A+B contract*—a *measured* minimum of layers actually executed **and** every other layer carries an explicit applicable/not-applicable verdict—otherwise the pipeline *refuses to close* and exits non-zero, a refusal that overrides even a valid human attestation. (4) The engine is turned on itself and the audit trail is shipped, including a different-vendor review that overturned an earlier version of our own central empirical claim. We make **no detection-superiority claim**; the contribution is the closure discipline itself, and a per-run *disclosure record*—which layers ran, which were adjudicated out and why, what independence was attested, whether closure was reached—that a judge cannot silently violate. This is a machine-checked instance of the judge-deployment disclosure standard JUDGE is convening [4].

1 The problem: self-certification, not inaccuracy

Evaluation validity is a property of a judge *in a system*, not in isolation [4]. A well-calibrated judge can fail systematically once its output gates a safety decision or feeds back into training. The specific failure we target is *self-certification*: a judge that treats its own stability as validity and its own agreement as independence. Norman et al. [1] supply the empirical warrant—across 21 judges from nine providers and roughly 541,000 individual judgments, reliability and validity are dissociated—but a diagnosis is not a mechanism. If a biased-but-consistent judge can stamp its own output “validated,” everything downstream inherits an error the system cannot see. The question we take up is not *how accurate is the judge* but *can the judge be stopped from closing its own loop*.

2 The protocol

The system is an orchestrator that spawns adversarial LLM sub-agent roles (an oracle that cites facts and emits no verdicts; a verifier that recomputes and, where code is runnable, *executes* it; a

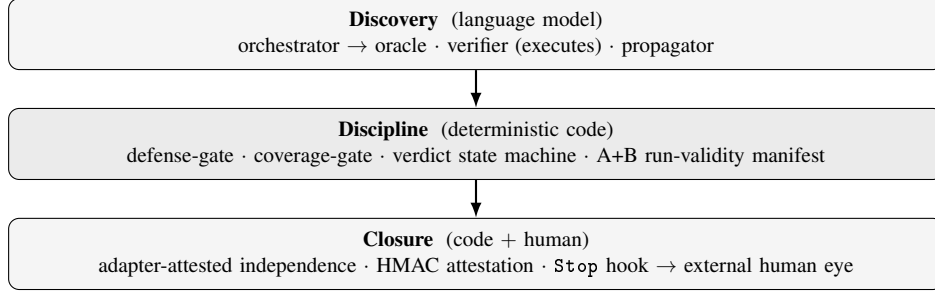


Figure 1: Three bands. Fallibility lives in the language-model band; *discipline and trust live in ordinary code*. The core assigns verdicts and decides closure; it can refuse, but it cannot self-certify.

propagator that constructs the concrete sequence violating a guarantee) and hands their findings to a *deterministic core* that assigns verdicts and decides completion. The orchestrator coordinates; it does not judge (Figure 1). Four invariants live in that core.

I1 — Non-closure is cryptographic. The completion state machine reaches `VALIDATED` only on a valid HMAC of the ledger digest under an operator key held outside the model’s reach. A bare “human-approved” token that does not verify closes nothing. A `Stop hook` re-verifies the HMAC in the operator’s environment and, if a `VALIDATED` does not verify, downgrades the ledger on disk. The model authors the payload but not the key: it cannot forge its own validation.

I2 — Independence is attested, not claimed. Different-vendor review is credited (independence level 3) only from the identity the *calling adapter* reports. A different-vendor label present only in the payload is recorded as `CROSS_MODEL_CLAIMED` and buys no independence credit. This directly answers the self-preference / correlated-blind-spot facet: a second model prompted adversarially by the same vendor is not counted as an independent draw.

I3 — The A+B run-validity contract. A pipeline that silently skips layers can still print a confident verdict; the danger is a run that *looks* complete because no one counted what did not happen. A run is admitted only if **(A)** a measured minimum of required layers actually executed—measured from each layer’s own output, not from self-report—and **(B)** every other layer carries an explicit `RAN/NOT_APPLICABLE/MISSING` status with justification. The manifest computes `run_validity`; if it is not `VALID`, completion is forced to `INVALID_RUN`, which *overrides every other state, including a valid human VALIDATED*, and the process exits non-zero. Closure cannot be bought by signing an incomplete process. The required minimum was itself *measured* across ten runs over eight artifact classes, not assumed—and the measurement corrected an earlier over-inclusion (one role frozen as required was returned to optional once two classes reached their defects without it).

I4 — Self-audit, shipped. The engine is run against its own artifacts and papers and the ledgers are committed. This is not decoration: a same-vendor self-audit (independence level 1) located four defects in our own closure guarantees; a later different-vendor review (level 3) overturned the original headline empirical claim and forced two guarantees from convention into cryptographic code; and the measurement instrument of I3, when first run, found three real defects *in itself*. We report these because a system whose value is adversarial honesty must show the audits that caught it.

Each invariant is a test in a 173-test suite run in CI across Python 3.10–3.13, including an end-to-end step that *fails if the pipeline ever prints VALIDATED without a human*.

```

run_validity : VALID # A+B contract satisfied
completion  : EXTERNAL_REVIEW_PENDING # no attested external identity → not
validated
layers      : triage=RAN oracle=RAN verifier=RAN propagator=RAN governor=RAN
              reasoner=NOT_APPLICABLE # not selected by triage
              deep_causal=NOT_APPLICABLE # not selected by triage
              external_auditor=NOT_APPLICABLE # not selected by triage
source_grade : {primary:2, institutional:0, generalist:0, unknown:0}

```

Figure 2: A per-run *disclosure record* — verbatim output of `run_core.py` on a complete run. It states which layers ran, which were adjudicated out and *why*, that the A+B contract holds (VALID), and that closure was *not* reached because no external identity was attested. This is a machine-checked instance of the disclosure JUDGE is convening; the pipeline refuses to close when the record is incomplete.

Reproducibility and artifact. The system is an MIT-licensed engine with four reproducible benchmarks and the 173-test suite above. From a clone, `python3 -m unittest discover -s tests and reproduce.py --strict` on each benchmark recompute every quantitative claim to the digit; the disclosure record of Figure 2 is the verbatim output of `run_core.py`. An anonymized copy of the repository is provided for review so that reviewers can clone it and run the gates, the refusal, and the cryptographic-closure check themselves; the public repository is disclosed on acceptance.

3 Relation to existing work

Reliability vs. validity of judges. [1] is the motivation and the missing-enforcement counterpart: they measure that judges are reliable-without-valid and recommend a “Minimum Viable Validation Protocol,” but a recommendation is advisory—a judge can decline it and certify itself anyway. We make the most dangerous consequence, self-certification, structurally impossible in code. We do not re-measure judge bias.

Adversarial defect discovery. The closest adversarial system, Refute-or-Promote [2], shares the adversarial kill-mandate and a cross-model critic and demonstrates them at scale (a 31-day campaign, ~171 candidates, ~79% killed before disclosure, real CVEs and accepted ISO C++ defect reports). We claim no priority on those ideas. The difference is the objective: Refute-or-Promote optimizes *precision of discovery* with a human orchestrator and no enforced non-closure—its cross-model step *improves* reliability but nothing *forbids* a confident close. We optimize *trustworthiness of closure*: judging is code, independence must be adapter-attested, closure is cryptographic, and an under-run is refused non-bypassably. A better *finder* versus a *finder that cannot lie about having finished*; the two are complementary.

Adversarial robustness of judges. Work on prompt-injection attacks against judges [3] asks whether a judge can be *fooled from outside*. We ask the orthogonal question—whether a judge can be *stopped from certifying itself from inside*—and enforce the answer in code rather than in a prompt.

JUDGE’s disclosure template. The workshop’s community deliverable is a judge-deployment disclosure standard (provenance, deployment context, known failure modes, human validation) [4]. Our execution manifest and A+B record are a *machine-checked instance* of such a disclosure: emitted per run, refused when incomplete.

4 Limitations

We make no detection-superiority claim; a small-sample study behind this system (seven real errors, one run per model nature) is a descriptive dissociation, not a recall result, and is reported as such in the companion systems paper. The cryptographic non-closure is only as strong as the operator’s key hygiene—it contracts forgery from a payload field to an out-of-band secret, it does not eliminate it. Real cross-vendor independence depends on an adapter the operator must wire up; absent it, the external-auditor role is theatre, and the code says so. And the whole apparatus is a *survivor*: every internal audit sits at independence level ≤ 3 in a single lab and validates nothing—by construction, only an external human eye closes the loop. That is the protocol’s own thesis applied to itself.

References

- [1] J. D. Norman, M. U. Rivera, and D. A. Hughes. Reliability without Validity: A Systematic, Large-Scale Evaluation of LLM-as-a-Judge Models Across Agreement, Consistency, and Bias. arXiv:2606.19544, 2026.
- [2] A. Agarwal. Refute-or-Promote: An Adversarial Stage-Gated Multi-Agent Review Methodology for High-Precision LLM-Assisted Defect Discovery. arXiv:2604.19049, 2026.
- [3] N. Maloyan and D. Namiot. Adversarial Attacks on LLM-as-a-Judge Systems: Insights from Prompt Injections. arXiv:2504.18333, 2025.
- [4] JUDGE @ NeurIPS 2026. Can We Trust the Judge? Workshop call for papers and failure taxonomy. <https://judge2026.github.io/>, 2026.